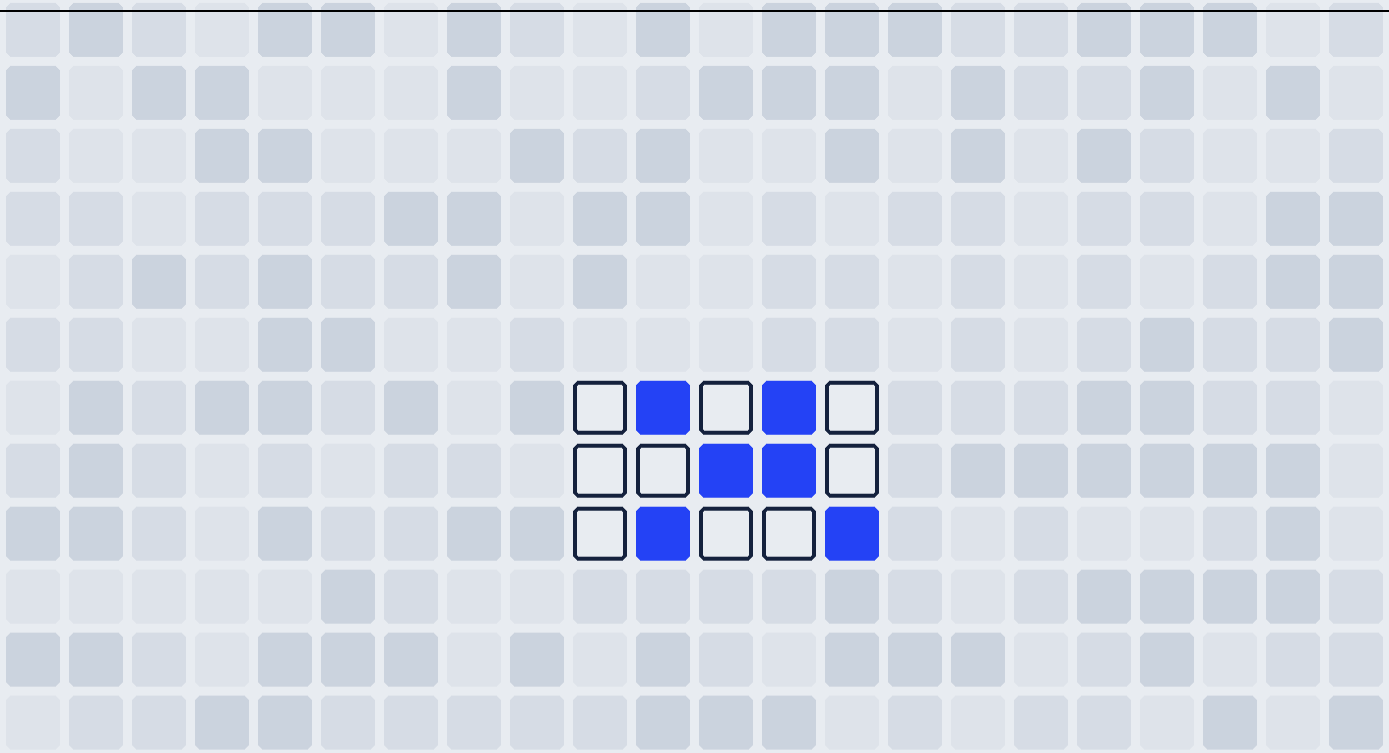


Freed memory Old data the next request could still reach

PHP2Shell

A case study on two critical PHP vulnerabilities

CVE-2026-109618 and CVE-2026-109619

Animesh Pradhan

October 24, 2026

This is an independent research case study and a record of my own understanding of a public security incident, based on the sources listed in the report. It is for learning only and is not security, legal or technical advice. I have not tested these vulnerabilities, and nothing here gives exploit steps. There may be mistakes. These are my personal views, not those of any employer, university or client.

PHP2Shell: A Case Study on Two Critical PHP Vulnerabilities

Contents

PHP2Shell: A Case Study on Two Critical PHP Vulnerabilities 1
1. Introduction 3
2. Background: what is PHP and why is it everywhere 3
3. What happened (timeline) 3
4. How the bugs work (simple version) 4
5. How it was found 4
6. Who was affected 5
7. Response 5
8. Mitigation 5
9. What I learned 6
10. Sources 6

1. Introduction

This case study is about PHP2Shell, the nickname for two serious security bugs in PHP itself. They are tracked as **CVE-2026-109618 and CVE-2026-109619**. I picked this case because it affected a huge number of websites within a few days, and because the disclosure did not go as planned.

2. Background: what is PHP and why is it everywhere

PHP is a programming language that runs on the server side of a website. *If a website is a restaurant, PHP works in the kitchen*: it takes the request, does the work, and sends back the page. PHP has been around since the mid-1990s.

It stayed popular because it is cheap, easy to learn, almost every hosting company supports it, and big tools like WordPress, Drupal and Joomla are built on it. Wiz estimates PHP powers roughly 70 percent of websites worldwide, though that does not mean 70 percent are vulnerable to these attacks [2]. Newer options like Node.js came later, but many companies stayed on PHP because rewriting a working website costs time and money. That is why one bug in PHP itself can touch so many sites at once.

3. What happened (timeline)

- **October 20:** Details of the two bugs leaked online before the planned disclosure. Someone using the name petecooper is named as the leaker [3]. A post on Hacker News was taken down, but screenshots spread. PHP warned that an emergency release was coming, and watchTowr confirmed the bugs were real and reproduced one of them [5].
- **October 21 (early):** Cloudflare's firewall had been blocking exploit attempts since 3:02 am UTC [6].
- **October 21:** PHP released the fixes, which had been scheduled for 14:00 UTC [11]. Fixed versions are 8.5.12, 8.4.27, 8.3.36 and 8.2.35 [1].
- **October 21:** Many organisations in Europe took websites offline before the patch was out [7].
- watchTowr said the researchers had planned the disclosure for Thursday, October 22 [5], so the leak pushed everything a day earlier.

4. How the bugs work (simple version)

PHP gives each web request some temporary memory, then cleans it up when the job is done, then moves on to the next request. Both bugs happen in edge cases of this handling. I am explaining it at a high level only.

Bug 1, CVE-2026-109618: If PHP runs out of memory while handling a request, some internal data can be left half set up. Then the cleanup step uses that bad data. If an attacker can influence what is in it, PHP can end up running the attacker's instructions [1][8]. PHP's default memory limit is 128MB, I believe.

Bug 2, CVE-2026-109619: A PHP worker can keep a reference to memory that was already freed. The next request handled by the same worker may use that leftover state before PHP resets it [1]. Things like fatal errors and timeouts that an attacker can trigger are part of this [8].

The result: An attacker can run commands on the server with just a web request and no login. This is called remote code execution (RCE). In the confirmed attack on mod_php, it happens before the website's own code runs, so a site's own login page does not protect it [8].

Not a backdoor. Nobody planted this. It is an accidental coding bug.

Not easy everywhere. VulnCheck found the exploit works reliably on ARM64, and on x86 only when ASLR (memory randomisation) is turned off [9]. Even so, exploitation attempts were reported soon after [9][10].

5. How it was found

Cloudflare says the bugs were disclosed by **Calif.io, Wiz, OpenAI and Quarkslab** [6]. Dark Reading credits Calif [11]. Researchers used AI models as a helper, but one analysis says it was humans who led the work and it was not found by AI alone [12]. Wiz said that frontier AI models lowered the barrier to turning a long-standing flaw into a working exploit [13].

Cloudflare says all servers running PHP 5.4 or later are affected [6]. PHP 5.4 came out around 2012, which is where I get the idea that the bug is old. I do not know exactly when the bug was introduced.

6. Who was affected

- Any server running an unpatched PHP: anything before 8.2.35, 8.3.36, 8.4.27 or 8.5.12 [1]. PHP 8.1 and older no longer get upstream fixes [14].
- In Europe, the Netherlands, Finland and France temporarily took public websites offline [7]. Belgium's foreign ministry and a Danish municipal IT company also took services down, and Wikimedia took some systems offline [7]. The Charité hospital in Berlin went offline too [15].
- Agencies did not agree on the danger. Cloudflare said it saw early attacks, while several national agencies said they had seen none [7].

7. Response

- **PHP:** Shipped an emergency release a day early.
- **Cloudflare:** Turned on managed firewall rules automatically for customers using its WAF [6]. Its CTO called this only a stopgap and told people to patch [10].
- **Vendors:** Vercel received advance notice and deployed firewall rules before public disclosure [1].
- **Security firms:** Published detection ideas and rules, like watching for PHP starting unexpected processes [8].

8. Mitigation

1. Find every PHP deployment: servers, containers, hosting, old test machines.
2. Update to a fixed version. PHP says there is no complete workaround, so updating is the only safe choice [8].
3. Updating WordPress, themes or plugins alone does not fix this, because the problem is in PHP underneath [2].
4. For old unsupported versions, move to a supported PHP release. That is not always a full rebuild, but it can need testing. Until then use a WAF, isolate the site, or take it offline.
5. Watch for PHP spawning odd processes and for out-of-memory errors in logs [8].
6. If a site was exposed while unpatched, check for signs of compromise after updating.

9. What I learned

- The leak made things worse, but a Huntress researcher said it also pushed defenders to prepare earlier [11].
- The bug was in the platform, not in the websites' own code, so even careful developers were exposed.
- Old technology that nobody wants to rewrite becomes a security risk over time.
- AI is shortening the time between a bug being found and an exploit being built [13].

10. Sources

1. Vercel, php2shell security bulletin (quotes PHP's advisory): https://vercel.com/kb/bulletin/php2shell-security-bulletin
2. Newsweek, PHP vulnerability coverage (Wiz estimate, US status, Automattic): https://www.newsweek.com/php-vunerability-php2shell-issue-websites-hack-server-12601225
3. Health-ISAC bulletin: https://www.aha.org/system/files/media/file/2026/10/h-isac-vulnerability-bulletins-tlp-white-two-critical-remote-code-execution-vulnerabilities-in-php-cve-2026-109618-cve-2026-109619-10-21-2026.pdf
4. Huntress, rapid response: https://huntress.com/blog/php2shell-leaked-critical-php-vulnerabilities
5. PPC Land, Cloudflare blocks php2shell exploits: https://ppc.land/cloudflare-blocks-php2shell-exploits-9-hours-before-php-flaws-went-public/
6. The Next Web, European governments pull websites offline: https://thenextweb.com/news/europe-governments-websites-offline-php-security-flaws
7. Picus Security, PHP2Shell explained: https://www.picussecurity.com/resource/php2shell-the-two-critical-php-rce-flaws-cve-2026-109618-cve-2026-109619-explained
8. Tanium, responding to PHP2Shell: https://www.tanium.com/blog/php2shell-responding-to-cve-2026-109618-and-cve-2026-109619
9. Itdaily, PHP2Shell exploited ahead of patch: https://itdaily.com/blogs/software/php2shell-bug-php/
10. Dark Reading, PHP2Shell bugs: https://www.darkreading.com/application-security/php2shell-bugs-red-alert-security-industry

11. SmartScope, who found PHP2Shell and the AI role: https://smartscope.blog/en/blog/php2shell-ai-really-find-php-vulnerability-roles-openai-2026/
12. Help Net Security, PHP2Shell patched: https://helpnetsecurity.com/2026/10/21/php2shell-php-vulnerabilities-patched-cve-2026-109618-cve-2026-109619
13. Level Up Coding, how to patch PHP2Shell: https://levelup.gitconnected.com/php2shell-two-critical-php-flaws-are-being-exploited-heres-how-to-patch-today-add61aa15d10
14. blue News, php2shell security vulnerability: https://www.bluewin.ch/en/digital/security-vulnerability-threatens-millions-of-websites-what-does-that-mean-li.3643831